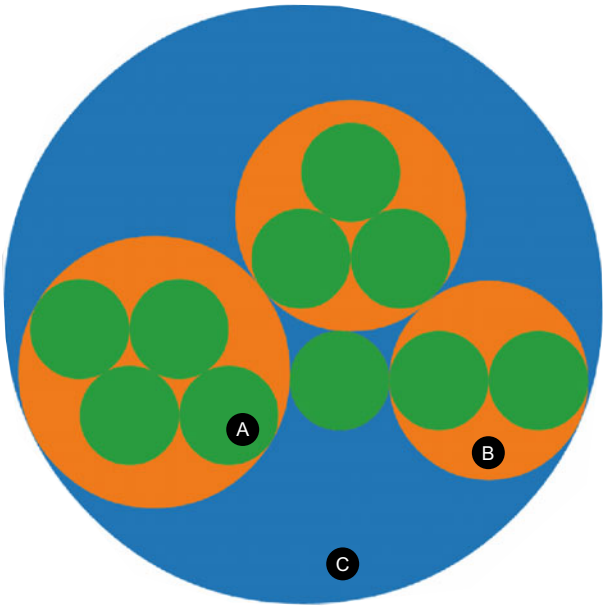


Figure 5.10 Each tweet is represented by a green circle (A) nested inside an orange circle (B) that represents the user who made the tweet. The users are all nested inside a blue circle (C) that represents our "root" node.

hierarchical parent of that object. You can see the hierarchical relationship. As with all layouts, the pack layout expects a default representation of data that may not align with the data you're working with. Specifically, pack expects a JSON object array where the child elements in a hierarchy are stored in a children attribute that points to an array. In examples of layout implementations on the web, the data is typically formatted to match the expected data format. In our case, we would format our tweets like this:

```
{id: "All Tweets", children: [
{id: "Al's Tweets", children: [{id: "tweet1"}, {id: "tweet2"}]},
{id: "Roy's Tweets", children: [{id: "tweet1"}, {id: "tweet2"}]}
...
```

But it's better to get accustomed to adjusting the accessor functions of the layout to match our data. This doesn't mean we don't have to do any data formatting. We still need to create a root node for circle packing to work (what's referred to as "All Tweets" in the previous code). But we'll adjust the accessor function `.children()` to match the structure of the data as it's represented in `nestedTweets`, which stores the child elements in the `values` attribute. In the following listing, we also override the `.value()` setting that determines the size of circles and set it to a fixed value, as shown in figure 5.10.

Listing 5.3 Circle packing of nested tweets data

```
var nestedTweets = d3.nest().key(function (el) {
        return el.user;
    }).entries(incData);

var packableTweets = {id: "All Tweets", values: nestedTweets};
```

Puts the array that d3.nest creates inside a "root" object that acts as the top-level parent